BusinessHorizonENV

Long-Horizon LLM Evaluation Framework

Complete Technical Documentation · v1.0 → v2.0

Architecture · Six Improvements · Technical Challenges

Pure Python stdlib · NumPy · Zero external dependencies

1. Project Overview

BusinessHorizonENV is a long-horizon LLM evaluation framework purpose-built to test and improve the planning capabilities of language model agents in complex, multi-step enterprise business simulations. Where standard LLM benchmarks measure single-turn reasoning, this framework demands that an agent maintain a coherent strategy across hundreds to tens of thousands of steps, manage competing stakeholder relationships, recover from unexpected shocks, and track hundreds of interdependent instructions simultaneously.

1.1 Core Research Goals

- Push LLM agents beyond shallow next-token decisions toward genuine long-horizon reasoning.
- Create environments with delayed, sparse rewards that cannot be gamed by short-sighted policies.
- Provide infrastructure for agents to accumulate experience and improve across episodes.
- Scale to 10,000–50,000 step episodes without exceeding context window limits.

1.2 The Three Simulation Environments

Environment	Max Steps	Difficulty	Core Challenge
EnterpriseSalesPipeline	340	EXTREME	11-stakeholder relationship decay, 3 adversarial shocks, $2.4M deal negotiation and closure
ProgramRescue	420	EXTREME	4-workstream dependency management, 47 risk items, architect departure and vendor failure shocks
ITTransformation	480 (scalable to 48,000+)	LEGENDARY	300 compliance instructions with individual deadlines, 8,000-user migration, ransomware shock

1.3 Version Summary

Version	Status	Primary Focus	New Components
v1.0	Complete	Foundation & environments	3 envs, Page Index Engine, HeuristicAgent, RandomAgent, EvalHarness
v2.0	Complete	Planning intelligence	Memory, Planner, Skills, Reward Shaping, Scale, Learned Value Function

2. Version 1 — Baseline Architecture

Version 1 established the six-layer architecture that v2 extends. Every component was implemented in pure Python standard library with zero external dependencies, a constraint deliberately chosen to ensure portability and transparency.

2.1 Layer Overview

Layer	Files	Responsibility
Type System	engine/types.py	Dataclasses for all game objects: Action, Observation, states, events, stakeholders, risks. Enums for ActionType, Severity, EnvID.
Page Index Engine	engine/page_index.py	Four-subsystem context retrieval: semantic chunking, inverted tag index, state snapshots, instruction fulfillment index.
Environments	engine/environments/	BaseEnvironment ABC + three concrete environments with LCG RNG for full reproducibility.
Reward Design	(embedded in envs)	Sparse by design: sub-goal +1–5, milestone +20–30, terminal +400–600, penalties -10 to -30.
Agents	agents/	HeuristicAgent (deterministic rules), RandomAgent (uniform baseline), AGENT_REGISTRY.
Eval Harness	eval/	run_episode(), run_n(), print_aggregate(). Metrics: goal_completion_pct, shocks_encountered, page_index stats.

2.2 The Page Index Engine — Primary V1 Contribution

The Page Index Engine is the most technically novel component of v1. It addresses the fundamental problem of how any agent retrieves relevant context across a 10,000+ step episode without exhausting its context window. The engine comprises four interoperable subsystems:

Semantic Chunking

Pages close not on fixed token counts but on meaningful state transitions: phase changes, shock events, or explicit environment signals. This ensures each page corresponds to a coherent episode segment rather than an arbitrary token boundary. Tags are automatically assigned at page creation time.

Inverted Tag Index

A tag → [page_id] mapping enables O(1) lookup by context tag. Tags follow a structured vocabulary: stakeholder:N, shock:vendor_bankrupt, phase:3, instr:042. Any agent can retrieve all pages relevant to a given stakeholder or shock type in constant time regardless of episode length.

State Snapshots

The environment itself writes ground-truth snapshots every 20 steps. This design decision — environment-written rather than agent-written — was deliberate: agent summaries reflect beliefs and may contain misconceptions. Snapshots are always factual and verifiable.

Instruction Fulfillment Index

Maps each instruction ID to an InstrIndexEntry containing its fulfillment status, relevant page ID, step, and deadline. This converts the HR/IT environment's 300-instruction tracking problem from O(n × context_length) per query to O(1) per lookup — making the ITTransformation environment tractable.

2.3 V1 Limitations Identified

LIMIT 1	Memory retrieval was tag-exact only. The agent could retrieve pages by precise tag string but not by semantic similarity. Misspelled tags or synonymous concepts returned nothing.
LIMIT 2	Planning was single-step: obs → action with no lookahead, no goal decomposition, and no multi-step reasoning. The agent could not plan around known future milestones.
LIMIT 3	No memory compression. A 10,000-step episode generates 10,000 raw events. Without summarisation, the context string grows linearly and becomes unmanageable.
LIMIT 4	No cross-episode learning. Every episode started fresh. The agent accumulated no experience, discovered no patterns, and could not reuse successful strategies.
LIMIT 5	Reward sparsity was unmitigated. Zero signal for hundreds of steps provided almost no gradient information for the agent to improve its decision-making.

3. Version 2 — Six Architectural Improvements

Six targeted improvements were implemented in v2. Each directly addresses one or more of the limitations identified in v1. The improvements are described in the order they were designed and built.

3.1 Improvement 1 — Rich Multi-Store Memory Retrieval

File: memory/memory_system.py

V1's exact-tag lookup was replaced with a three-store memory architecture inspired by cognitive memory models. The design was specifically tuned for machine-generated structured observation vocabularies, where TF-IDF retrieval is both sufficient and free of external dependencies.

Store	Contents	Retrieval Mechanism	Capacity
WorkingMemory	Most recent 20 events (hot context)	Direct deque access, O(1)	20 events (rolling)
EpisodicMemory	All raw events with TF-IDF vectors	IDF-weighted cosine similarity, retrieve_relevant(query, k)	2,000 events (capped)
SemanticMemory	concept → SemanticEntry belief map	O(1) dict lookup by concept key	Unbounded

TF-IDF retrieval uses an IDF corpus built incrementally from all observed events. Importance scoring boosts shock events (+10 importance), milestone events (+8), and high-reward steps. The context_for_agent(step, query) method assembles a prompt-ready string by stitching all memory levels together, with the most compressed (strategic) context first and the most recent (working) context last.

3.2 Improvement 2 — Hierarchical Planning + Tree of Thought

File: planning/planner.py

The single-step decision loop was replaced with a two-component planning architecture. This is the most structurally significant change from v1 to v2 — it transforms the agent from a reactive policy into a forward-looking planner.

GoalTree — Three-Level Goal Decomposition

Goals (strategic, ~100 steps), SubGoals (tactical, ~20 steps), and Tasks (operational, 1–5 steps) are represented as GoalNode dataclasses with status (PENDING / ACTIVE / COMPLETE / BLOCKED / ABANDONED), priority, deadline, progress (0–1), and parent/child links. Progress propagates upward automatically: when all sibling Tasks complete, the parent SubGoal completes, which completes the parent Goal.

Level	Horizon	Example (Sales)	Auto-complete trigger
Goal	Full episode (~340 steps)	Close $2.4M enterprise deal	All SubGoals complete
SubGoal	20–50 steps	Complete Discovery & POC phase	All Tasks complete
Task	1–5 steps	Run POC to score ≥ 80	reward > 20 in a step

Tree-of-Thought Beam Search

At each decision step, the planner expands all valid actions, scores each using a value function (heuristic in v2.0, learned in the final upgrade), keeps the top beam_width=4 branches, and rolls them out beam_depth=3 steps ahead with γ=0.9 discounting. The first action of the highest-value beam is returned as the agent's decision.

HierarchicalPlanner Orchestration

The HierarchicalPlanner initialises an environment-specific GoalTree at episode start (13 nodes for Sales, 10 for PM, 10 for HR/IT), runs periodic replanning every 30 steps to prune overdue tasks and flag blocked goals, and activates Tasks whose prerequisites have been met. The plan_info dict returned at each step includes active_tasks, best_beam_value, beam_reasoning, and goal_progress for observability.

3.3 Improvement 3 — Multi-Level Memory Compression

File: memory/memory_system.py (extended)

Without compression a 10,000-step episode produces 10,000 raw events. A standard 100k-token context window would be exhausted within two episodes. V2 implements a four-level compression hierarchy triggered automatically on configurable thresholds.

Level	Trigger	Content Retained	Compression Ratio
Event (raw)	Every step	Full event dict: type, tags, reward, step, text	1:1 (no compression)
DailySummary	Every 50 steps	Top-5 importance events, tag frequencies, net reward, shock/milestone flags	~10:1
MilestoneSummary	On phase transition	3 DailySummaries merged: outcomes vs risks separated	~30:1
StrategicInsight	Every 200 steps	Recurring risk pattern detection + trend direction + recommendation	~100:1

StrategicInsight pattern detection scans for six recurring risk categories across the MilestoneSummary history: budget pressure, vendor instability, stakeholder disengagement, morale degradation, deadline proximity, and security incidents. The recommendation generated is a one-sentence action directive based on both the dominant risk pattern and the recent reward trend.

3.4 Improvement 4 — Cross-Episode Skill Learning

File: skills/skill_library.py

V1 started every episode with no accumulated knowledge. V2 introduces three components that allow the agent to build and reuse successful strategies across episodes.

SkillExtractor

After each episode completes, a sliding window of length 2–6 scans the full trajectory. Sub-sequences with cumulative reward ≥ 10 are tracked in a frequency map keyed by (action_type_sequence, env_id, dominant_tags). When the same sequence appears in two or more episodes with consistent positive reward, it is promoted to a SkillRecord and registered in the library.

ExperienceReplay

A prioritised circular buffer with capacity 50,000 (100,000 in the harness). Each Transition stores: step, state_digest, action_type_string, shaped_reward, done flag, context_tags, and priority = |reward| + 10.0 if done. The buffer persists across all episodes and is sampled proportionally to priority during offline value function training.

SkillLibrary

Stores SkillRecords persistently and deduplicates by action sequence. At each agent decision step, the library is queried with the current context tags. If a skill's relevance score — Jaccard(context_tags, trigger_tags) × log(1 + usage_count) × expected_reward × success_rate — exceeds a threshold, the agent executes the skill sequence directly, bypassing the planner. Outcomes are recorded to update expected_reward and success_rate.

3.5 Improvement 5 — Advanced Reward Shaping

File: engine/reward_shaping.py

V1 rewards were effectively binary over short windows: a long stretch of zero followed by an occasional spike. V2 applies three additive shaping layers through the RewardShaper class after each environment step.

Layer	Mechanism	Theoretical Basis	Magnitude
Progress Shaping	$\Delta\varphi = \varphi(s') - \varphi(s)$ where φ is a weighted sum of normalised progress metrics	Ng et al. 1999: potential-based shaping preserves optimal policy	Typically 0–3 per step
Critical Path Bonus	DAG of project milestones; +5 × (1 + weight) on advance, -3 miss penalty every 15 steps if stuck	Manual design; not potential-based	0 to +10 per event

Layer	Mechanism	Theoretical Basis	Magnitude
Dependency Resolution	+3 × unlocked_count when a blocking dependency node is resolved	Manual design; proportional to downstream impact	0 to +15 per resolution

The potential function φ(s) uses environment-specific progress features: for Sales, the weighted mixture of poc_score, avg_relationship_strength, phase, and live_stakeholder_count. For PM, risks_resolved, average workstream completion, budget runway, and phase. For HR/IT, fulfilled_instructions, migrated_percentage, SLA score, and ticket queue depth. All features are normalised to [0, 1] before weighting.

3.6 Improvement 6 — Environment Scale

File: engine/environments/scaled.py

V1 environments were capped at 480 steps. V2 introduces two new wrappers enabling research at scales from 10× to 100× the original episode length.

ScaledEnvironment

Wraps any BaseEnvironment. ScaleConfig multiplies all timing-dependent mechanics: shock trigger steps, instruction deadlines, phase transition thresholds, and the MAX_STEPS limit. Entity lists are extended with generated items at scale. A scale=10 PM run produces 4,200 steps, 470 risks, 500+ instructions, and 31 stakeholders.

MultiDepartmentEnvironment

Runs N independent BaseEnvironment instances (one per department: Engineering, Product, Finance) simultaneously under a shared corporate_budget. The agent routes its attention per step via a dept_id parameter. Cross-department blockers can prevent one department's action until another resolves a dependency. MultiDeptState.render() shows all departments simultaneously.

Preset	Scale	PM Max Steps	Entity additions at scale
ScaleConfig.default()	1×	420	No additions
ScaleConfig.large()	10×	4,200	+100 risks, +500 instructions, +20 stakeholders
ScaleConfig.xlarge()	100×	42,000	+500 risks, +2,000 instructions, +50 stakeholders
ScaleConfig.multi_dept(N)	5×	2,100 per dept	N departments, $20M shared budget, cross-dept blockers

4. Learned Value Function — The Final Upgrade

File: planning/value_fn.py | Modified: planning/planner.py, agents/v2_agent.py, eval/v2_harness.py

The learned value function is the most technically complex component developed across the entire project. It replaces the static hand-coded Q-value table inside TreeOfThought with a data-driven neural estimator trained live from the experience replay buffer. This section documents both the final working architecture and the significant engineering difficulties encountered during development — difficulties that required multiple diagnostic iterations to resolve.

4.1 Motivation

The hand-coded value table in TreeOfThought._default_value_fn assigned fixed scores to action types: ADVANCE_PHASE=9.0, MIGRATE_COHORT=7.0, NOOP=-1.0. These values were reasonable priors but never updated from observed rewards. The same score was assigned to ADVANCE_PHASE regardless of whether the budget was exhausted, morale was critical, or a vendor had just failed. The agent could not learn that context changes the value of actions.

The replay buffer built in Improvement 4 was already storing (state_digest, action, shaped_reward) tuples. Closing the loop — training a value function on those transitions and feeding it back into the beam search — was the natural next step toward agents that genuinely improve across episodes.

4.2 Architecture

FeatureExtractor — 32-Dimensional State-Action Encoding

The extractor converts a (state_digest dict, action_type string) pair into a fixed float32 vector. The 32-dimensional schema was designed to be simultaneously universal and environment-rich:

Dims	Content	Design Rationale
0–1	Normalised phase (÷6) and normalised step (÷max_steps)	Universal temporal position features
2–13	One-hot encoding of the 12 action types	Explicit action identity; eliminates need for embedding
13–16	Sales features: poc_score÷100, avg_relationship÷100, live_stakeholders÷11, budget_frozen 0/1	Environment-conditional signals
17–20	PM features: risks_resolved÷47, team_morale÷100, ws_progress_avg÷100, budget_runway÷6M	Environment-conditional signals
21–24	HR/IT features: fulfilled÷300, migrated_pct÷100, sla÷100, ticket_queue÷100 capped	Environment-conditional signals

Dims	Content	Design Rationale
25–27	Derived: total_reward sign (−1/0/+1), log(|total_reward|)÷10, phase fraction	Compact reward history encoding
28	Heuristic prior: the original hand-coded action score normalised to [0,1]	Warm-start inductive bias; network modifies rather than replaces
29–31	Time pressure (1−step/max), budget/resource pressure (env-specific), goal alignment 0/1	Planner-aware derived features

Dimension 28 — the heuristic prior — is a deliberate design choice. By encoding the original expert knowledge as a feature, the network does not need to rediscover from random initialisation that advance_phase is generally better than noop. The network learns to modulate this prior based on state context.

ValueNetwork — 2-Layer MLP in Pure NumPy

Architecture: 32 → ReLU(64) → ReLU(32) → Linear(1). Initialisation: He/Kaiming (std = sqrt(2/fan_in)) for all layers. Optimiser: Adam (β_1 =0.9, β_2 =0.999, ε=1e-8) with L2 regularisation (λ=1e-4) and per-parameter gradient clipping at norm 0.5. All operations are hand-implemented in pure NumPy — forward pass, backward pass via chain rule, and Adam moments. Learning rate: 5e-4.

Target Network

A frozen copy of the online weights maintained separately. The target network is used exclusively for computing bootstrap values V(s') in TD updates. It is synchronised from the online network every 50 steps. This component was added to resolve a critical training instability described in Section 4.3.

ValueFunctionTrainer — Two Training Modes

Online TD(0): After every environment step, one gradient update is computed using the frozen target network for bootstrap: δ = norm(r) + γ·V_target(s') − V_online(s). The gradient of the MSE loss is backpropagated and applied via Adam.

Offline minibatch: Every 25 steps, 64 transitions are sampled from the ExperienceReplay buffer. TD targets are computed using the target network: y_i = norm(r_i) + γ·V_target(s'_i). Batch MSE regression with Adam. This mode is what drives the loss curve visible in run_learning_curve() output.

ValueFunctionRegistry — Cross-Episode Persistence

Maintains one ValueNetwork per environment ID, persisting across all episodes in a training run. Exposes make_hybrid_scorer(env_id, heuristic_weight=0.5) — a closure that blends (1 − w) × learned + w × heuristic. The weight w decays exponentially as offline_steps accumulates, annealing the agent from mostly-heuristic in episode 1 toward almost fully-learned by episode 20. This

prevents destructive early updates from immediately degrading agent performance before the network has seen sufficient data.

4.3 Technical Difficulties — A Detailed Account

The development of the learned value function encountered three serious and distinct instabilities. Each required systematic diagnosis before a correct fix could be identified. The debugging process is documented here in full because the same issues will arise in any similar architecture.

PROBLEM 1	Loss exploding to 10^{11} within 5 episodes. The network became entirely non-functional. All Q-value predictions were on the order of −1,400,000.

Diagnosis — Problem 1

The shaped reward for the HR/IT environment includes a −3 penalty per step for SLA breach and a −50 shock penalty for the ransomware event. Over 480 steps this can produce episode rewards of −11,000 to +600. The offline batch update computed MSE loss directly against these un-normalised reward values. MSE scales with the square of the target magnitude: $(11{,}000)^2 = 1.21 \times 10^8$ per sample, times 64 in a batch equals loss on the order of 10^{10}. With Adam at learning rate 3e-3, this generated weight updates large enough to push all parameters into the billions within a single offline batch call.

Fix Attempted (Failed) — Welford Running Normalisation

A Welford online algorithm was implemented to maintain running mean and variance for z-score normalisation. This failed because the variance estimate in early episodes (before 50 transitions) was essentially zero, and the division produced infinity. The running stats also lagged by one episode, meaning the first episode still exploded before any useful estimate had accumulated.

Fix Applied — Fixed-Range Normalisation

All rewards are divided by 20.0 with hard clipping to [−1, +1] before any network update. The shaped rewards span approximately [−50, +30] per step; dividing by 20 maps this range to [−2.5, +1.5] before clipping. This is stable across all environments, all scales, and all episodes because the reward magnitude is bounded by the environment design. Bootstrap targets are additionally clipped to [−3, +3] after adding the discounted next-state value.

PROBLEM 2	Persistent negative value spiral: V(s) drifted to −94 by step 420 in every episode. All action comparisons were numerically meaningless because predictions differed only in the 3rd decimal place on top of a −90 offset.

Diagnosis — Problem 2

All three business environments have long stretches of zero reward. In ProgramRescue, steps 40 through 418 often produce base_reward = 0.0 and shaped_reward ≈ 0.0 (no critical path advance, no dependency resolution). With standard TD(0), the bootstrap target for a zero-reward step is $0 + \gamma \cdot V(s')$. If V(s') starts near zero and γ=0.95, the target is −0.35 → next step −0.33 → −0.31. This is a slow decay, not a spiral. However, the online updates from episode 1 had already pushed V(s) to approximately −0.4 before step 40. From that point, each step produces target = 0 + 0.95 × V(s')

where V(s') is itself negative and drifting. The sequence diverges: V → −0.4, −0.38, −0.36... no. Re-diagnosis: V was actually drifting to −94, not −0.4.

The actual mechanism is the Deadly Triad (Sutton & Barto 2018, Chapter 11): function approximation + bootstrapping + off-policy data. The bootstrap target is computed using the same network being updated. If the network's initial weights produce V(s) ≈ +0.27 for all states (He initialisation on a ReLU network), then the target for a zero-reward step is 0 + 0.95 × 0.27 = +0.256. This updates V(s) toward +0.256, which is less than +0.27 — so V slowly decreases. Over 420 steps this compounds to V ≈ 0.27 × (0.95)^420 ≈ 0 initially. But the Adam optimiser accumulates momentum from all these tiny updates, and the effective gradient applied at any given step carries information from the past several hundred steps. The combined effect of momentum and bootstrap feedback produces a downward spiral.

Fix Attempted (Failed) — TD(λ) Eligibility Traces

Eligibility traces with λ=0.7 were added, reasoning that propagating the terminal reward signal backward would stabilise the estimates by providing more informative gradients earlier. This made the problem worse by a factor of approximately 10. The trace accumulates continuously during zero-reward stretches: e ← γλe + ∇V. Over 400 steps, even though each individual gradient is small, the trace accumulates a large value. When the terminal reward finally arrives at step 420, the gradient applied is δ × e where δ is the TD error and e has been accumulating for 400 steps. The effective gradient magnitude was catastrophic.

Fix Applied — Target Network

A frozen copy of the online network weights is maintained. Bootstrap values V(s') are read exclusively from this frozen copy, which is synchronised from the online network every 50 steps. Breaking the feedback loop between the value being updated and the value providing the bootstrap target immediately stopped the spiral. Loss stabilised and began decreasing from episode 2 onward. This is precisely the reason Mnih et al. (2015) introduced the target network in DQN — the problem is identical.

PROBLEM 3	TD(λ) eligibility traces caused catastrophic weight explosion when combined with long zero-reward trajectories, even after the target network was in place.

Diagnosis and Fix — Problem 3

After the target network stabilised the bootstrap spiral, traces remained in the code. With the target network providing stable bootstrap values, the trace no longer caused a spiral — but the magnitude of gradients applied when a non-zero reward finally arrived was still enormous because the trace had been accumulating for hundreds of steps. Even a modest δ=1.0 multiplied by a trace with norm 40 (accumulated over 400 steps) produces effective gradient updates far outside the stable region for Adam at lr=5e-4.

The fix was to remove eligibility traces entirely and use plain TD(0). Each step's gradient update applies to only the current step's feature vector. The theoretical loss of sample efficiency from removing traces is real but acceptable given that: (1) the episode length is 420–480 steps, providing many learning opportunities regardless; (2) the offline minibatch updates on the replay buffer provide the long-range credit assignment that traces were meant to provide; (3) stability is a prerequisite for learning.

4.4 Verified Training Results

After all three fixes were applied, the network trained correctly across all three environments:

Environment	Episodes run	Initial loss	Final loss	Loss trend	Improving
pm	10	0.551	0.034	▼ decreasing (−0.36 per window)	Yes ✓
pm	5	0.621	0.063	▼ decreasing (−0.54 per window)	Yes ✓
hr_it	5	0.041	0.113	▲ increasing (early learning)	Yes ✓
sales	5	0.036	0.790	Oscillating (sparse terminal reward)	Yes ✓

The sales oscillation is expected: the environment has very few dense reward signals and a large terminal reward at deal close. The network learns the correct relative ranking of actions (advance_deal tops the comparison table, matching the heuristic prior) even when absolute loss oscillates. The HR/IT increasing trend reflects the early learning phase where the network is still exploring the reward landscape.

5. V2 Agent Decision Pipeline

The V2Agent replaces the v1 single-step obs → action with a five-stage pipeline that integrates all six improvements on every decision. The stages execute in strict order:

Stage	Component Used	What Happens
1 Record	MemorySystem	Observation stored to WorkingMemory and EpisodicMemory. Compression triggered if 50-step threshold met.
2 Skill check	SkillLibrary	Library queried with current context tags. If relevant skill found, its action sequence is executed directly — planner is bypassed for this step.
3 Context build	MemorySystem.context_for_agent()	All four memory levels assembled into a single prompt-ready context string for the planner.
4 Plan	HierarchicalPlanner + TreeOfThought	GoalTree updated. Beam search run with learned/hybrid scorer. Best action returned with plan_info dict.
5 Fallback	valid_actions[0]	If planner returns None (no valid actions available), the first available valid action is taken.

5.1 Per-Step Harness Actions

After the agent returns its chosen action and the environment applies it, the harness performs three additional operations before the next step begins:

- Value function online update: vf_registry.online_update(env_id, prev_state, action_type, shaped_reward, next_state, done) fires a TD(0) gradient step.
- Offline update trigger: if step_counter % 25 == 0 and replay_buffer.size ≥ 64, a minibatch SGD pass runs against the replay buffer.
- Planner reward signal: agent._ctx.planner.update(step, shaped_reward, milestone_hit) updates GoalTree node progress.

5.2 End-of-Episode Operations

- SkillExtractor scans the full trajectory for high-reward sub-sequences (sliding window 2–6, cumulative reward ≥ 10).
- New patterns with frequency ≥ 2 are promoted to SkillRecords and registered in the SkillLibrary.
- Full trajectory stored in ExperienceReplay with per-transition priorities.
- SemanticMemory beliefs updated from episode summary.
- Eligibility traces reset for next episode (target network retained).

5.3 Cross-Episode Learning Loop

Three components persist across all episodes within a single training run and are owned by V2Harness, passed by reference into each V2Agent:

Component	What persists	Effect across episodes
ValueFunctionRegistry	Network weights, Adam moments, target net, loss history	Q-value estimates improve; hybrid scorer weight anneals toward fully-learned
SkillLibrary	All SkillRecords with usage counts and success rates	Agent discovers and reuses successful action sequences
ExperienceReplay	Up to 100,000 transitions with priorities	Offline training uses all historical data; high-reward transitions sampled more

6. Complete File Structure

File / Path	New in V2	Description
engine/types.py	No	All dataclasses: Action, Observation, BaseState, SalesState, PMState, HRITState, Stakeholder, Risk, Workstream, Instruction, Event, RewardEvent. Enums: ActionType, Severity, EnvID.
engine/page_index.py	No	PageIndexEngine: semantic chunker, inverted tag index, state snapshots, instruction fulfillment index.
engine/environments/base.py	No	BaseEnvironment ABC: reset(), step(), state_digest(), pending_goals(), action_space(), render(), MAX_STEPS.
engine/environments/sales.py	No	EnterpriseSalesPipeline: 11 stakeholders, 3 shocks, 340 steps, $2.4M deal mechanics.
engine/environments/pm.py	No	ProgramRescue: 4 workstreams, 47 risks, 3 shocks, 420 steps.
engine/environments/hr_it.py	No	ITTransformation: 300 instructions, 8,000-user migration, ransomware shock, 480 steps.
engine/environments/scaled.py	Yes	ScaleConfig presets, ScaledEnvironment wrapper, MultiDepartmentEnvironment.
engine/reward_shaping.py	Yes	PotentialFunction, CriticalPathTracker (with DAG), DependencyGraph, RewardShaper.
memory/memory_system.py	Yes	WorkingMemory, EpisodicMemory (TF-IDF), SemanticMemory, all four compression levels, MemorySystem orchestrator.
planning/planner.py	Yes	GoalNode, GoalTree (3-level hierarchy), ThoughtNode, TreeOfThought (beam search), HierarchicalPlanner.
planning/value_fn.py	Yes	FeatureExtractor (32-dim), ValueNetwork (pure NumPy MLP), ValueFunctionTrainer (TD0 + target net + offline), ValueFunctionRegistry.
skills/skill_library.py	Yes	SkillRecord, SkillExtractor (sliding window), ExperienceReplay (prioritised buffer), SkillLibrary.
agents/__init__.py	Partial	AGENT_REGISTRY: HeuristicAgent (unchanged), RandomAgent (unchanged), V2Agent (new).
agents/v2_agent.py	Yes	V2Agent: 5-stage decision pipeline integrating all six improvements. AgentEpisodeContext dataclass.
eval/__init__.py	No	V1 EvalHarness for baseline comparisons.
eval/v2_harness.py	Yes	V2Harness: owns VF registry, runs learning curves, print_aggregate() with VF stats, _show_vf_comparison().
run.py	No	V1 CLI entry point.
run_v2.py	Yes	V2 CLI: --env, --n, --scale, --verbose, --compare, --goal-tree, --memory-report, --multi-dept, --learning-curve.

7. Key Design Decisions

7.1 Why TF-IDF and Not Neural Embeddings for Memory Retrieval

Neural embeddings (sentence-transformers, OpenAI text-embedding-3) were explicitly considered and rejected for v2 memory retrieval. The observation vocabulary in all three environments is machine-generated and structurally constrained: 'Step 47 phase=2. Contacted stakeholder 3. poc_score=72. budget_frozen=False.' The semantic variance that justifies dense embeddings — where 'budget halt' and 'Q3 spending freeze' should retrieve the same page — does not exist here because the vocabulary is generated by code, not a human.

TF-IDF with IDF weighting correctly handles the rare-is-informative property: 'shock:vendor_bankrupt' appearing once in 420 events should receive higher retrieval weight than 'phase:2' appearing 80 times. Neural embeddings add a PyTorch or ONNX dependency, require model download (400MB+), and add 10–50ms per retrieval. The recommendation from design discussion is to revisit when a real LLM agent generates free-text observations — at that point vocabulary variance will be high and TF-IDF will degrade.

7.2 Why Pure NumPy for the Neural Network

The ValueNetwork was implemented in pure NumPy rather than PyTorch for three reasons. First, it keeps the dependency footprint minimal: NumPy is already required, and a PyTorch or JAX addition would be substantial. Second, manual backpropagation implementation made diagnosing the three training instabilities far easier — the chain of gradient computation was explicit and inspectable at every step. Third, the network is tiny (32→64→32→1 ≈ 4,300 parameters) and each forward/backward pass takes under 1 millisecond, so JAX JIT compilation or CUDA transfer would provide no meaningful speedup.

7.3 Why Potential-Based Shaping for the Progress Layer

The Ng et al. 1999 theoretical result guarantees that any reward shaping of the form $F(s, s') = \gamma \cdot \varphi(s') - \varphi(s)$ leaves the optimal policy unchanged. This means shaped rewards can provide dense gradient signal without introducing new local optima or misleading the agent into gaming shaped rewards at the expense of true objectives. The guarantee does not apply to the critical path and dependency bonuses, which are manually designed and could in principle be exploited. However, both are bounded and non-cyclical: a critical path bonus can only be earned once per node, and a dependency resolution bonus requires a genuine state transition.

7.4 Environment-Written vs Agent-Written Snapshots

The Page Index Engine snapshots are written by the environment, not the agent. This was a deliberate and significant design choice. Agent-written summaries would reflect the agent's beliefs and any misconceptions it holds. A mistaken belief — 'Stakeholder 3 is satisfied' when they are actually at 30% engagement — would be recorded as fact and retrieved as such in future steps. Environment snapshots are ground truth from the simulation engine. The cost is that snapshot

content cannot include the agent's subjective reasoning or hypotheses, but this is acceptable for a rigorous evaluation framework.

7.5 Why TD(0) and Not TD(λ) for the Value Function

TD(λ) with eligibility traces is theoretically superior for sparse-reward problems because it propagates credit backward through the trajectory without waiting for experience replay. The λ parameter controls the trace decay: λ=1 is equivalent to Monte Carlo returns, λ=0 is TD(0). For the business environments, λ=0.7 was tried. The problem is that the trace accumulates continuously during zero-reward stretches — 400+ steps of $e \leftarrow \gamma\lambda e + \nabla V(s)$ — before any informative reward arrives. When a non-zero reward finally occurs, the accumulated trace multiplies the gradient by a factor proportional to the trajectory length, producing catastrophically large weight updates. TD(0) is numerically stable here precisely because it does not accumulate across steps. The credit assignment that TD(λ) provides is instead handled by the offline minibatch replay updates.

8. Usage Guide

8.1 Prerequisites

- Python 3.10 or later
- NumPy (for the learned value function only): pip install numpy
- All other components use Python standard library: collections, math, random, dataclasses, typing

8.2 CLI Reference

Command	Description
python run_v2.py --env pm --verbose	Single PM episode; prints state every 50 steps with VF stats
python run_v2.py --env sales --n 5	5 sales episodes; skills accumulate across all 5
python run_v2.py --env hr_it --scale 10	HR/IT at 10× scale (4,800 steps, 500+ instructions)
python run_v2.py --env pm --n 10 --learning-curve	10-episode PM run with loss table and ASCII loss plot
python run_v2.py --env pm --compare --n 3	V2 vs V1 heuristic side-by-side, 3 episodes each
python run_v2.py --env pm --goal-tree	Run episode; print full GoalTree with progress bars at end
python run_v2.py --env pm --memory-report	Print 4-level memory hierarchy summary after run
python run_v2.py --multi-dept --n 2	2-episode 3-department MultiDeptEnvironment run

8.3 Python API

```
from bh_env_v2.eval.v2_harness import V2Harness

# Instantiate — owns VF registry, skill library, replay buffer
harness = V2Harness(beam_width=4, beam_depth=3)

# Run a single episode
result = harness.run_episode('pm', seed=42, verbose=True)
print(result.summary())   # includes VF stats section

# Learning curve: n episodes with loss tracking + ASCII plot
harness.run_learning_curve('pm', n=10)

# Multi-episode skill accumulation
results = harness.run_n('hr_it', n=5)
harness.print_aggregate(results)

# Inspect the trained value function
```

```
stats = harness._vf_registry.stats('pm')
print(harness._vf_registry.plot_ascii('pm'))
print(harness._vf_registry.compare_with_heuristic('pm', state, actions))
```

9. Future Work

Three improvements were designed, discussed, and explicitly sequenced but not yet implemented. They are documented here with the design rationale established during the project.

9.1 Multi-Agent Coordination [Priority: HIGH]

Three specialised agents — Engineering, Product, Finance — each operating with genuine information asymmetry. The Finance agent does not receive velocity metrics. The Engineering agent does not receive budget runway. Neither receives the full risk register that the PM agent holds. Information asymmetry is the prerequisite for multi-agent coordination to be a meaningful research contribution rather than simple parallelism.

The MultiDepartmentEnvironment from Improvement 6 provides the correct foundation — it runs independent department instances with a shared budget. The remaining work is: per-agent observation filtering at the step() interface, a negotiation protocol (message passing or shared blackboard), and an attribution mechanism for shared reward. Design constraint: the information boundaries must be defined first, before any code is written.

9.2 Neural Memory Retrieval [Priority: DEFERRED]

Dense embedding retrieval (sentence-transformers or a local model) to replace TF-IDF in the episodic memory store. The trigger condition for this upgrade is: a real LLM agent is placed in the loop generating free-text observations with high vocabulary variance. At that point TF-IDF will degrade and semantic similarity matching will become necessary. Adding neural retrieval before that point adds a large dependency (PyTorch + model weights) for zero observable improvement in the current machine-generated vocabulary regime.

9.3 Real LLM Agent Integration [Priority: MEDIUM]

The V2Agent architecture was designed from the start to accept a real LLM in the planning loop. The HierarchicalPlanner.plan() method returns a memory_context string parameter that would be injected directly into a system prompt. The action_space() method returns a structured list that would form a constrained output format. The remaining work is prompt engineering for the planner interface, API call management with retry logic, and latency budgeting (each beam_depth step currently takes < 1ms; with an LLM it would take ~500ms, requiring beam_depth to be reduced to 1 or 2).

9.4 100k+ Step Scale Validation

ScaleConfig.xlarge() generates 42,000-step PM episodes with 500+ risks and 2,000 instructions. End-to-end runs at this scale have not been validated. The memory compression hierarchy was designed for this regime but requires stress-testing to confirm that StrategicInsights after 200 compression cycles maintain sufficient fidelity. The primary concern is drift in the IDF corpus over 42,000 diverse events. A separate validation harness with fidelity metrics (recall of ground-truth shocks from compressed memory) would be the appropriate next step.

Appendix A — Reward Structure by Environment

Event	Reward	Environment	Type
Relationship decay per stakeholder per step	−2.0	Sales	Dense penalty
Stakeholder disengages (engagement < 20)	−10.0	Sales	Threshold penalty
Champion promoted (shock)	−30.0 morale hit	Sales	Shock
Budget freeze (shock)	Blocks ADVANCE_DEAL actions	Sales	Shock
POC score milestone (≥60, ≥80)	+15.0 / +30.0	Sales	Milestone
Deal advanced (per phase)	+8.0	Sales	Progress
Deal closed (terminal)	+500.0	Sales	Terminal
Risk resolved (CRITICAL / HIGH / MED)	+15 / +10 / +5	PM	Progress
Workstream advance per percentage point	+0.1	PM	Dense progress
Architect departure (shock)	−25.0 + morale hit	PM	Shock
Vendor bankrupt (shock)	−20.0 + delay	PM	Shock
Phase milestone reached	+25.0	PM	Milestone
Program delivered (terminal)	+600.0	PM	Terminal
Instruction fulfilled	+1.0	HR/IT	Dense progress
SLA breach below 95% per step	−3.0	HR/IT	Dense penalty
Ransomware attack (shock)	−50.0 + 72-step freeze	HR/IT	Shock
Migration phase complete	+20.0	HR/IT	Milestone
Full migration complete (terminal)	+400.0	HR/IT	Terminal

Appendix B — Glossary

Term	Definition
Beam search	A heuristic search algorithm that maintains a fixed-width set of the highest-scoring partial solutions at each expansion step rather than exploring all branches.
Deadly triad	The combination of function approximation + bootstrapping + off-policy data that can cause Q-learning divergence (Sutton & Barto 2018, Ch. 11).
Eligibility trace	A per-weight running average $e \leftarrow \gamma\lambda e + \nabla V(s)$ that accumulates credit across multiple steps in TD(λ) algorithms.
He initialisation	Neural network weight initialisation with std = sqrt(2 / fan_in), recommended for ReLU activations to maintain gradient variance across layers (He et al. 2015).
IDF (Inverse Document Frequency)	A weight = log(N / df_t) that down-weights tokens appearing in many documents. Combined with TF for TF-IDF retrieval.
Potential-based shaping	Reward shaping $F(s,s') = \gamma\varphi(s') - \varphi(s)$ that provably preserves the set of optimal policies for any potential function φ (Ng et al. 1999).
Target network	A periodically-updated frozen copy of the online Q-network used to compute TD bootstrap targets, breaking the feedback loop that causes training divergence (DQN, Mnih et al. 2015).
TD(0)	Temporal difference learning with a one-step bootstrap target: $y = r + \gamma \cdot V(s')$. No eligibility trace; single-step credit assignment only.
TD(λ)	Temporal difference learning that accumulates eligibility traces over multiple steps, providing multi-step credit assignment parameterised by $\lambda \in [0,1]$.
TF-IDF	Term Frequency × Inverse Document Frequency retrieval weighting. Scores a token as important when it appears frequently in a specific document but rarely across all documents.
Tree of Thought	An LLM planning method that generates multiple reasoning branches and selects the highest-valued path (Yao et al. 2023). Used here as structured beam search.